HALADÓ PROGRAMOZÁSI TECHNIKÁK 1

Beadandó feladat

Csomagküldő szolgáltatás kezelő rendszere

TANTÁRGY	DOKUMENTUM	VERZIÓ
Haladó programozási technikák 1	Hallgatói specifikáció	1.0

Megjegyzés: Ez a dokumentum a beadandó funkcionális és technikai elvárásait rögzíti. A beadási forma, az értékelési rendszer és a beszámoló részletei külön dokumentumban szerepelnek.

DOKUMENTUMSTRUKTÚRA

Tartalomjegyzék

A dokumentum fő fejezetei és alfejezetei.

1. A feladat

2. Technológiai keretek

3. Ügyfelek kezelése

4. Új csomag feladása

5. Csomagtípusok

5.1. Normál csomag

5.2. Törékeny csomag

5.3. Hűtést igénylő csomag

6. A csomag állapotai és életciklusa

7. Nyomkövetési történet

8. Kézbesítési kísérletek

8.1. Sikeres kézbesítés

8.2. Sikertelen kézbesítés

9. Csomagok keresése

9.1. Keresés nyomkövetési azonosító alapján

9.2. Keresés feladó alapján

9.3. Keresés címzett alapján

9.4. Keresés állapot alapján

9.5. Keresés csomagtípus alapján

9.6. Keresés feladási időszak alapján

10. Riportok és kimutatások

10.1. Csomagok száma állapotonként

10.2. Csomagtípus-statisztika

10.3. Öt legaktívabb feladó

10.4. Sikertelen kézbesítések okai

10.5. Időszaki feladási kimutatás

10.6. Speciális kezelést igénylő aktív csomagok

11. Hibás bevitel és üzleti hibák

12. Adatbázis és Entity Framework Core

13. A program felépítése

14. A félév során tanult technikák

15. A feladat határai

16. Külső segítség és AI-eszközök

17. Mikor tekinthető késznek a beadandó?

1. A feladat

Egy csomagküldő szolgáltatás belső nyilvántartó rendszerét kell elkészítened C# nyelven.

A rendszer feladata:

- a szolgáltatás ügyfeleinek nyilvántartása;
- új csomagok feladása;
- a csomagok feldolgozási folyamatának követése;
- a kézbesítési folyamat és a sikertelen kézbesítések kezelése;
- a csomagok nyomkövetési történetének megőrzése;
- keresési lehetőségek biztosítása;
- statisztikák és kimutatások készítése.

A program hagyományos konzolos asztali alkalmazás. Nem webalkalmazást, webszervert vagy API-t kell készítened.

A program belépési pontját megvalósíthatod top-level statementekkel, vagy használhatsz hagyományos `Program` osztályt és `Main` metódust is. A két megoldás az értékelés szempontjából egyenértékű.

A beadandóhoz kapsz egy működő induló projektet. Ebben:

- a konzolos menürendszer alapja már működik;
- a hibás menüpontválasztás nem állítja le a programot;
- a fő funkciókhoz előkészített menüpontok tartoznak;
- a szükséges Entity Framework Core NuGet-csomagok már telepítve vannak.

A kiinduló főmenü a következő funkciócsoportokat tartalmazza:

1. Ügyfelek kezelése
2. Új csomag feladása
3. Csomag feldolgozása
4. Csomag nyomkövetése
5. Csomagok keresése
6. Riportok

0. Kilépés

A menürendszert módosíthatod vagy további almenükkel egészítheted ki, de a specifikációban szereplő valamennyi funkciónak elérhetőnek kell lennie belőle.

Az egyes menüpontok mögötti működés, az alkalmazás domainmodellje, üzleti logikája és adatbázis-kezelése a te feladatod.

Az induló `Program.cs` kommentekkel is jelzi majd, hogy az egyes menüpontoknál hol érdemes meghívni a saját üzleti komponenseidet. A teljes üzleti logikát azonban ne közvetlenül a menürendszert kezelő kódba írd.



Gondold végig

A program belépési pontjához tartozó kód elsősorban a felhasználóval kommunikáljon: kérjen be adatot, indítsa el a megfelelő műveletet, majd jelenítse meg annak eredményét. Ha itt kezded ellenőrizni a csomag állapotát, lekérdezni az adatbázist és kiszámítani a riportokat is, valószínűleg túl sok felelősséget tettél egy helyre.

2. Technológiai keretek

Az alkalmazás .NET-es C# konzolos programként készüljön.

Az adatok tartós tárolására kötelezően:

- Entity Framework Core;
- SQLite adatbázis

használandó.

A projektnek tartalmaznia kell legalább egy működő EF Core migrációt, amelyből az alkalmazás adatbázisa létrehozható.

Az adatbázis-kezelésnél a tantárgyban tanult EF Core funkcionalitás használata elegendő:

- `DbContext`;
- `DbSet<T>`;
- adatok létrehozása;
- adatok módosítása;
- adatok lekérdezése;
- egyszerű entitáskapcsolatok;
- alap Fluent API konfiguráció;
- migráció;
- LINQ-alapú lekérdezések.

Nem követelmény:

- explicit tranzakciókezelés;
- repository pattern;
- Unit of Work;
- raw SQL;
- concurrency kezelés;
- haladó EF Core mapping;
- adatbázis-teljesítményhangolás.

3. Ügyfelek kezelése

A rendszer nyilvántartja a csomagküldő szolgáltatás ügyfeleit.

Egy ügyfél egyaránt lehet csomag feladója és címzettje.

Minden ügyfélhez tartozzon:

- egy belső egyedi azonosító;
- név;
- e-mail-cím;
- telefonszám.

A név, az e-mail-cím és a telefonszám kötelező adat, egyik sem lehet üres.

Az e-mail-cím csak akkor fogadható el, ha legalább egy `@` karaktert tartalmaz, és az nem az első vagy utolsó karakter.

Az ügyfélkezelő menüből az alábbi műveletek legyenek elérhetők:

- új ügyfél rögzítése;
- összes ügyfél listázása;
- egy ügyfél részletes adatainak megjelenítése;
- ügyfél nevének, e-mail-címének vagy telefonszámának módosítása;
- az ügyfél által feladott csomagok megjelenítése;
- az ügyfélnek címzett csomagok megjelenítése.

Ügyfél törlését nem kell megvalósítani. Ennek oka, hogy egy ügyfél korábbi csomagok feladójaként vagy címzettjeként már része lehet a nyilvántartott előzményeknek.

4. Új csomag feladása

Új csomag feladásakor a felhasználó először válassza ki a már nyilvántartott ügyfelek közül:

- a feladót;
- a címzettet.

Ugyanaz az ügyfél egy csomagnál nem lehet egyszerre feladó és címzett.

Ezután meg kell adni a kézbesítési címet:

- irányítószám;
- település;
- közterület neve;
- közterület jellege;
- házszám;
- opcionális további címadat.

Az irányítószám, település, közterület neve, közterület jellege és házszám kötelező.

A kézbesítési cím közvetlenül az adott csomaghoz tartozzon. Ha egy ügyfél adatai később megváltoznak, egy már feladott csomag kézbesítési címe ne változzon meg automatikusan.

Ezután ki kell választani a csomag típusát, és meg kell adni a csomag tömegét.

A tömeg kilogrammban értendő, pozitív szám kell legyen.

A csomag létrehozásakor a rendszer automatikusan:

- hozzon létre belső azonosítót;
- hozzon létre egyedi nyomkövetési azonosítót;
- rögzítse a feladás időpontját;
- állítsa a csomag állapotát `Registered` értékre;
- készítse el az első nyomkövetési bejegyzést.

A sikeres feladás után a létrehozott nyomkövetési azonosító jelenjen meg a felhasználónak.

A nyomkövetési azonosító formátuma:

> PKG-ÉÉÉÉ-XXXXX

ahol:

- `ÉÉÉÉ` a feladás éve;
- `XXXXX` öt számjegyből álló azonosító rész.

A pontos előállítási algoritmust neked kell kialakítanod, de két csomag nem rendelkezhet azonos nyomkövetési azonosítóval.

> 💡 **Gondold végig**
>
> Az egyediségre nemcsak a C# kódban érdemes támaszkodni. A nyomkövetési azonosító olyan adat, amelynek az adatbázisban sem szabad kétszer előfordulnia. Nézd meg, hogyan tudod ezt Fluent API segítségével is kifejezni.

5. Csomagtípusok

A rendszer pontosan háromféle csomagot kezel.

5.1. Normál csomag

A normál csomag nem igényel speciális kezelést.

A csomag tömege:

- nagyobb kell legyen 0 kg-nál;
- legfeljebb 30 kg lehet.

A normál csomaghoz a minden csomagra közös adatokon túl nem szükséges további speciális adatot tárolni.

5.2. Törékeny csomag

A törékeny csomag tömege:

- nagyobb kell legyen 0 kg-nál;
- legfeljebb 10 kg lehet.

Minden törékeny csomaghoz kötelező megadni:

- a törékenység szintjét;
- egy kezelési utasítást.

A törékenység szintje csak a következő három érték valamelyike lehet:

- `Low`;
- `Medium`;
- `High`.

A kezelési utasítás rövid szöveges információ, például a csomag elhelyezésére vagy mozgatására vonatkozó megjegyzés. Nem lehet üres.

A csomag részletes adatainak megjelenítésekor a törékenységi szint és a kezelési utasítás is jelenjen meg.

5.3. Hűtést igénylő csomag

A hűtést igénylő csomag tömege:

- nagyobb kell legyen 0 kg-nál;
- legfeljebb 20 kg lehet.

Minden ilyen csomaghoz meg kell adni:

- a megengedett minimális tárolási hőmérsékletet Celsius-fokban;
- a megengedett maximális tárolási hőmérsékletet Celsius-fokban.

A minimális értéknek kisebbnek kell lennie a maximális értéknél.

A rendszerben megadható hőmérsékleti tartomány:

```
-30 °C és +20 °C között
```

lehet.

Például a

```
+2 °C és +8 °C
```

tartomány érvényes, míg a

```
+10 °C és +4 °C
```

nem.

A csomag részletes adatainak megjelenítésekor a szükséges tárolási hőmérséklet-tartomány is jelenjen meg.

A rendszernek nem kell tényleges hőmérsékletmérést vagy szenzoradatokat kezelnie.

> 💡 **Gondold végig**
>
> A három csomagtípusnak sok közös tulajdonsága van, ugyanakkor a törékeny és a hűtött csomag további saját adatokat is tartalmaz. Gondold végig, hogyan modellezhető ez úgy az objektumorientált eszközökkel, hogy a közös részeket ne kelljen többször megírnod. Arra is figyelj, hogy a választott modellt később EF Core segítségével is perzisztálnod kell.

6. A csomag állapotai és életciklusa

Minden csomagnak pontosan egy aktuális állapota van.

A rendszer az alábbi állapotokat kezeli:

- `Registered` – Feladva;
- `Accepted` – Átvéve;
- `AtDepot` – Elosztóközpontban;
- `OutForDelivery` – Kiszállítás alatt;
- `DeliveryFailed` – Sikertelen kézbesítés;
- `Delivered` – Kézbesítve;
- `Returning` – Visszaküldés alatt;
- `Returned` – Visszaküldve.

Új csomag mindig `Registered` állapotban jön létre.

Az engedélyezett állapotátmenetek:

Aktuális állapot	Engedélyezett következő állapot
`Registered`	`Accepted`
`Accepted`	`AtDepot`
`AtDepot`	`OutForDelivery`
`OutForDelivery`	`Delivered`, `DeliveryFailed`
`DeliveryFailed`	`OutForDelivery`, `Returning`
`Returning`	`Returned`
`Delivered`	nincs
`Returned`	nincs

A `Delivered` és `Returned` végállapotok.

Ezekből a csomag további állapotba nem helyezhető.

A csomagfeldolgozó menüpontban a felhasználó először adja meg vagy válassza ki a kezelendő csomagot.

A program az aktuális állapot alapján csak az értelmes következő műveleteket ajánlja fel.

Például egy `Registered` csomagnál az „Átvétel rögzítése”, egy `AtDepot` csomagnál a „Kiszállítás megkezdése” művelet jelenjen meg.

A felhasználói felület által felajánlott lehetőségektől függetlenül az üzleti logika is ellenőrizze, hogy az állapotváltás valóban engedélyezett-e.



Gondold végig

Egy állapothoz egy vagy több megengedett következő állapot tartozik. Ezt hosszú `if-else` vagy `switch` szerkezettel is le lehet írni, de a félév során tanult kollekciók között van olyan, amely természetesen reprezentál ilyen kulcs–érték kapcsolatot.

7. Nyomkövetési történet

Minden sikeres állapotváltozáskor új nyomkövetési bejegyzést kell készíteni.

Egy nyomkövetési bejegyzés tartalmazza:

- az érintett csomagot;
- az új állapotot;
- a változás időpontját;
- opcionális megjegyzést;
- sikertelen kézbesítés esetén a sikertelenség okát.

Új csomag létrehozásakor automatikusan létrejön az első `Registered` bejegyzés.

Ha egy állapotváltás nem engedélyezett, akkor:

- a csomag aktuális állapota nem változhat meg;
- új nyomkövetési bejegyzés sem készülhet.

A „Csomag nyomkövetése” funkciónál a felhasználó nyomkövetési azonosítót ad meg.

A rendszer jelenítse meg:

- a csomag típusát;
- feladóját;
- címzettjét;
- kézbesítési címét;
- tömegét;
- feladási időpontját;
- aktuális állapotát;
- a csomagtípushoz tartozó speciális adatokat;
- a teljes nyomkövetési történetet időrendben.

A történet legrégebbi bejegyzése jelenjen meg először.

Például:

```
Csomag: PKG-2026-00125
Típus: Törékeny
Feladó: Kovács Anna
Címzett: Nagy Péter
Tömeg: 4,5 kg
Törékenység: High
Kezelési utasítás: Állítva szállítandó
Aktuális állapot: Delivered

Nyomkövetés:
2026.10.03. 08:14    Registered
2026.10.03. 12:37    Accepted
2026.10.03. 18:22    AtDepot
2026.10.04. 07:51    OutForDelivery
2026.10.04. 10:16    DeliveryFailed - RecipientNotAvailable
2026.10.05. 08:03    OutForDelivery
2026.10.05. 09:44    Delivered
```



Gondold végig

Az aktuális állapot és az állapottörténet két különböző célt szolgál. Az egyikből gyorsan megtudod, hol tart most a csomag, a másikból azt, hogyan jutott el oda. Ha mindkettőt tárolod, az állapotváltási műveletnek kell gondoskodnia arról, hogy összhangban maradjanak.

8. Kézbesítési kísérletek

Kézbesítési eredményt csak `OutForDelivery` állapotú csomagnál lehet rögzíteni.

A felhasználó két eredmény közül választhat:

- sikeres kézbesítés;
- sikertelen kézbesítés.

8.1. Sikeres kézbesítés

Siker esetén:

- a csomag állapota `Delivered` lesz;
- új `Delivered` nyomkövetési bejegyzés készül;
- a csomag életciklusa lezárul.

8.2. Sikertelen kézbesítés

Sikertelen kézbesítésnél az alábbi okok valamelyikét kell kiválasztani:

- `RecipientNotAvailable` – a címzett nem volt elérhető;
- `AddressNotFound` – a cím nem volt megtalálható;
- `RecipientRefused` – a címzett megtagadta az átvételt.

Szabad szöveges sikertelenségi ok nem használható.

Sikertelen kézbesítéskor:

- a csomag állapota `DeliveryFailed` lesz;
- a kiválasztott ok bekerül a nyomkövetési bejegyzésbe.

Egy csomaghoz legfeljebb három `DeliveryFailed` bejegyzés tartozhat.

Az első és második sikertelen próbálkozás után a felhasználó:

- új kézbesítési kísérletet kezdeményezhet, ekkor a csomag ismét `OutForDelivery` állapotba kerül;
- vagy elindíthatja a visszaküldést, ekkor a csomag `Returning` állapotba kerül.

A harmadik sikertelen kézbesítés után újabb kézbesítési kísérlet már nem indítható. Ekkor csak a visszaküldési folyamat kezdhető meg.

`Returning` állapotból a csomag `Returned` állapotba helyezhető.

A `Returned` állapot a csomag végállapota.



Gondold végig

A sikertelen kézbesítések számát már tartalmazhatja a csomag története. Mielőtt külön számláló property-t készítenél, vizsgáld meg, hogy a szükséges információ kiszámítható-e a már tárolt adatokból. Itt a LINQ is hasznos lehet.

9. Csomagok keresése

A keresési menüből külön-külön legyen elérhető az alábbi hat keresés.

9.1. Keresés nyomkövetési azonosító alapján

A felhasználó megad egy teljes nyomkövetési azonosítót.

A rendszer:

- megjeleníti a hozzá tartozó csomag alapadatait;
- vagy jelzi, ha ilyen csomag nem található.

9.2. Keresés feladó alapján

A felhasználó kiválaszt egy ügyfelet.

A rendszer jelenítse meg az általa feladott összes csomagot a feladás időpontja szerint csökkenő sorrendben.

9.3. Keresés címzett alapján

A felhasználó kiválaszt egy ügyfelet.

A rendszer jelenítse meg a neki címzett összes csomagot a feladás időpontja szerint csökkenő sorrendben.

9.4. Keresés állapot alapján

A felhasználó kiválaszt egy `PackageStatus` értéket.

A rendszer jelenítse meg az adott aktuális állapotban lévő csomagokat.

9.5. Keresés csomagtípus alapján

A felhasználó kiválasztja a három csomagtípus egyikét.

A rendszer jelenítse meg az adott típushoz tartozó csomagokat.

9.6. Keresés feladási időszak alapján

A felhasználó megad:

- egy kezdő dátumot;
- egy záró dátumot.

A kezdő dátum nem lehet későbbi a záró dátumnál.

A rendszer jelenítse meg az időszakban feladott csomagokat a feladás időpontja szerint növekvő sorrendben.

A több találatot adó listákban jelenjen meg legalább:

- nyomkövetési azonosító;
- csomagtípus;
- feladó neve;
- címzett neve;
- tömeg;
- aktuális állapot;
- feladás időpontja.

> 💡 **Gondold végig**
>
> Ezeknek a kereséseknek a jelentős része ugyanarra az alapműveletre épül: egy adathalmazból feltételek alapján választasz ki elemeket, majd rendezed őket. Mielőtt ciklusokkal kezded el felépíteni ezeket, nézd meg a tanult LINQ műveleteket.

10. Riportok és kimutatások

A riportok az SQLite adatbázis aktuális adataiból készüljenek EF Core és LINQ segítségével.

10.1. Csomagok száma állapotonként

Minden olyan állapot jelenjen meg, amelyhez legalább egy csomag tartozik.

Az eredmény tartalmazza:

- az állapotot;
- az adott állapotban lévő csomagok számát.

Az eredmény csomagszám szerint csökkenő sorrendben jelenjen meg.

10.2. Csomagtípus-statisztika

Mindhárom csomagtípushoz jelenjen meg:

- a csomagok száma;
- az összesített tömeg;
- az átlagos tömeg.

Az eredmény a csomagok száma szerint csökkenő sorrendben jelenjen meg.

10.3. Öt legaktívabb feladó

A rendszer jelenítse meg az öt legtöbb csomagot feladó ügyfelet.

Minden eredménysorban szerepeljen:

- az ügyfél neve;
- a feladott csomagok száma;
- a feladott csomagok össztömege.

Az eredmény a csomagok száma szerint csökkenő sorrendben jelenjen meg.

10.4. Sikertelen kézbesítések okai

A rendszer mindhárom sikertelenségi okhoz jelenítse meg:

- az előfordulások számát.

Az eredmény a gyakoriság szerint csökkenő sorrendben jelenjen meg.

10.5. Időszaki feladási kimutatás

A felhasználó megad egy kezdő és egy záró dátumot.

A rendszer jelenítse meg:

- az időszakban feladott csomagok számát;
- azok össztömegét;
- azok átlagos tömegét;
- majd magukat a csomagokat feladási időpont szerint növekvő sorrendben.

10.6. Speciális kezelést igénylő aktív csomagok

Jelenjen meg minden olyan törékeny vagy hűtést igénylő csomag, amely még nem `Delivered` és nem `Returned` állapotú.

Törékeny csomagnál jelenjen meg:

- a törékenység szintje;
- a kezelési utasítás.

Hűtést igénylő csomagnál jelenjen meg:

- a minimális tárolási hőmérséklet;
- a maximális tárolási hőmérséklet.

Az eredmény feladás időpontja szerint növekvő sorrendben jelenjen meg.



Gondold végig

A riportok elkészítéséhez szűrésre, csoportosításra, rendezésre, projekcióra és aggregációra is szükséged lesz. A `Where`, `GroupBy`, `Select`, `OrderBy`, `Count`, `Sum` és `Average` jellegű LINQ műveleteket éppen ilyen problémák megoldására tanultuk.

Nem szükséges először az adatbázis teljes tartalmát memóriába tölteni, majd ott elvégezni az összes szűrést.

11. Hibás bevitel és üzleti hibák

A program egyszerű hibás felhasználói input miatt nem állhat le.

Ha a program:

- számot vár, de nem számot kap;
- dátumot vár, de hibás dátumot kap;
- menüpontot vár, de nem létező menüpontot kap,

akkor jelenítsen meg hibaüzenetet, és kérje be újra az adatot.

Az üzleti szabályok megsértését külön kezeld.

Ilyen eset többek között:

- nem létező nyomkövetési azonosító használata;
- ugyanaz az ügyfél lenne a feladó és a címzett;
- nem engedélyezett állapotváltás;
- végállapotban lévő csomag módosítása;
- túl nagy tömegű csomag feladása;
- hiányzó törékenycsomag-adat;
- hibás hőmérséklet-tartomány;
- harmadik sikertelen kézbesítés után újabb kézbesítési kísérlet;
- kézbesítési eredmény rögzítése nem `OutForDelivery` állapotú csomagnál.

Az üzleti hiba ne állítsa le a programot. A felhasználó kapjon érthető visszajelzést, majd tudja tovább használni az alkalmazást.



Gondold végig

A hibás billentyűzetes bevitel és egy üzleti szabály megsértése nem ugyanaz. A félév során tanult exceptionkezelés különösen arra alkalmas, hogy az üzleti logika jelezzen egy problémát annak a programrésznek, amely már tudja, hogyan kell azt a felhasználónak megjeleníteni.

Nem szükséges minden üzleti hibához külön exceptionosztályt létrehozni. Ha azonban több hibát eltérően szeretnél kezelni vagy világosan elkülöníteni, saját exceptiontípus használata indokolt lehet.

A `finally` használata nem kötelező. Csak akkor alkalmazd, ha ténylegesen van olyan művelet, amelynek a `try` blokk sikerétől vagy sikertelenségétől függetlenül végre kell hajtódnia.

12. Adatbázis és Entity Framework Core

Az alkalmazás tartós adatait SQLite adatbázisban kell tárolni Entity Framework Core segítségével.

A program bezárása és újraindítása után a korábban rögzített adatoknak továbbra is elérhetőnek kell lenniük.

Tartósan meg kell őrizni legalább:

- az ügyfeleket;
- a csomagokat;
- a csomagtípus-specifikus adatokat;
- a csomagok állapotát;
- a nyomkövetési bejegyzéseket;
- a sikertelen kézbesítések okait.

A projektben neked kell elkészítened:

- a saját `DbContext` osztályt;
- a szükséges `DbSet<T>` property-ket;
- az entitások közötti kapcsolatokat;
- a szükséges alap Fluent API konfigurációt;
- legalább egy működő migrációt.

A nyomkövetési azonosító adatbázis-szinten is legyen egyedi.

A csomag és az ügyfél között két külön kapcsolat van:

- feladó;
- címzett.

Ezeket az EF Core modellben egyértelműen meg kell különböztetni.

Egy csomaghoz több nyomkövetési bejegyzés tartozhat, egy nyomkövetési bejegyzés viszont pontosan egy csomaghoz kapcsolódik.

> 💡 **Gondold végig**
>
> Az EF Core konvenciói sok egyszerű kapcsolatot automatikusan felismernek. Ebben a rendszerben azonban ugyanaz a `Customer` típus két külön szerepben kapcsolódik ugyanahhoz a csomaghoz. Ez olyan helyzet, ahol a tanult Fluent API különösen hasznos lehet.

Az adatbázis létrehozása migrációval történjen. Egy kézzel előre elkészített SQLite adatbázisfájl önmagában nem helyettesíti a migrációt.

---

13. A program felépítése

A teljes megoldást ne egyetlen `Program.cs` fájlban valósítsd meg.

Függetlenül attól, hogy top-level statementeket vagy hagyományos `Main` metódust használsz, a program belépési pontja és a konzolos menürendszer elsősorban:

- adatot kérjen be;
- indítsa el a megfelelő műveletet;
- jelenítse meg az eredményt;
- kezelje a felhasználónak szóló hibaüzenetek megjelenítését.

A következő feladatkörök logikája ne közvetlenül a menükezelő kódban legyen:

- ügyfelek üzleti kezelése;
- csomag létrehozása;
- állapotváltási szabályok;
- kézbesítési logika;
- adatbázis-műveletek;
- keresések;
- riportok.

A konkrét osztálystruktúrát és fájlszerkezetet nem írjuk elő.



Gondold végig

Ha ugyanazt a műveletet később egy másik felhasználói felületről is el szeretnéd indítani, mennyi kódot kellene átmásolnod a `Program.cs` fájlból? Ha a válasz az, hogy sokat, akkor érdemes lehet az üzleti logikát külön komponensbe szervezni.

14. A félév során tanult technikák

A feladat megoldása során több helyen természetesen használhatók a félév során tanult eszközök:

- exceptionök és `try-catch-finally`;
- `Dictionary<TKey,TValue>`;
- lambda kifejezések;
- LINQ;
- extension methodok;
- Entity Framework Core.

A specifikáció nem írja elő minden problémánál a konkrét programozási eszközt. A feladat része annak felismerése is, hogy egy adott problémához melyik megoldás illeszkedik jól.



Gondold végig

Ha több helyen szükséged van például annak eldöntésére, hogy egy `PackageStatus` végállapot-e, könnyen ugyanazt az ellenőrzést kezdheted el ismételni. A félév során tanult extension method lehetőséget adhat arra, hogy egy meglévő típushoz jól elnevezett, természetesen használható műveletet kapcsolj.

Nem az a cél, hogy minden tanult technikából mesterséges példát készíts. Olyan eszközt válassz, amelynek valódi szerepe van a megoldásban.

15. A feladat határai

A beadandó nem egy teljes futárszolgálati vállalatirányítási rendszer.

Nem kell megvalósítani:

- futárok nyilvántartását;
- járművek kezelését;
- depók részletes kezelését;
- útvonaltervezést;
- GPS-nyomkövetést;
- tényleges hőmérsékletmérést;
- térképes funkciókat;
- online fizetést;
- felhasználói bejelentkezést;
- jogosultságkezelést;
- webes vagy grafikus felületet;
- hálózati kommunikációt.

Ezek elkészítése nem szükséges a magas értékeléshez.

A beadandó értékét nem az extra funkciók száma, hanem a specifikációban megadott működés helyes, átgondolt és érthető megvalósítása határozza meg.

16. Külső segítség és AI-eszközök

A beadandó elkészítéséhez használhatsz:

- órai tananyagokat;
- hivatalos dokumentációkat;
- könyveket;
- internetes szakmai forrásokat;
- fejlesztői eszközöket;
- generatív AI-eszközöket.

AI-eszköz használata nem követelmény.

A feladat úgy került kialakításra, hogy a tantárgy tananyagai és nyilvánosan elérhető szakmai források segítségével is megoldható legyen.

A felhasznált segítségtől függetlenül a beadott program teljes tartalmáért te felelsz.

A beadandóhoz szóbeli beszámoló tartozik. A beszámolón képesnek kell lenned:

- bemutatni a program felépítését;
- elmagyarázni egy kiválasztott saját kódrészlet működését;
- indokolni az alkalmazott kollekciókat és nyelvi eszközöket;
- értelmezni saját LINQ lekérdezéseidet;
- elmagyarázni az EF Core adatmodellt és annak kapcsolatait;
- indokolni a hibakezelési döntéseidet;
- elmagyarázni, hogy egy megadott követelményváltozás a program mely részeit érintené.

A beszámolón nem lesz kötelező helyszíni programozási vagy kódmódosítási feladat.

17. Mikor tekinthető késznek a beadandó?

A beadandó funkcionálisan akkor tekinthető elkészültnek, ha az alkalmazás:

- lefordul és elindul;
- a konzolos menürendszerrel végig használható;
- ügyfeleket tud rögzíteni, módosítani és lekérdezni;
- mindhárom csomagtípust helyesen kezeli;
- a csomagtípusok speciális adatait is tárolja;
- érvényes csomagot tud feladni;
- SQLite adatbázisban tartósan tárolja az adatokat;
- kezeli az előírt csomag-életciklust;
- nem engedi az érvénytelen állapotváltásokat;
- minden állapotváltozást nyomon követ;
- kezeli a sikeres és sikertelen kézbesítést;
- a harmadik sikertelen kézbesítés után nem enged újabb kézbesítési kísérletet;
- kezeli a visszaküldési folyamatot;
- mind a hat előírt keresést megvalósítja;
- mind a hat előírt riportot elkészíti;
- hibás felhasználói inputtól nem áll le;
- az üzleti hibákat kezelhető módon jelzi;
- Entity Framework Core-t és SQLite-ot használ;
- tartalmaz működő EF Core migrációt;
- a lényegi üzleti logikát nem a program belépési pontjához tartozó kódban valósítja meg.

A specifikáció teljesítésére több különböző, szakmailag megfelelő programozási megoldás is létezhet.

A saját megoldásodat nemcsak elkészítened kell, hanem a szóbeli beszámoló során értened és indokolnod is.

A specifikáció teljesítésére több különböző, szakmailag megfelelő megoldás is létezhet. A beadandó értékelésénél a specifikáció helyes megvalósítása, a program szervezettsége és a megoldás megértése a meghatározó.